ML4NLP1 · TUTORIAL, WEEK 9

[CLS] [SEP] [ENT]

From BERT to GLiNER

1. Your questions on Ex4
2. [CLS] and [SEP] in BERT: answers to your questions
3. A short note on ModernBERT
4. How GLiNER works
5. Notebook: GLiNER, GLiNER2 and NuExtract

Papers and figures: References at the end of the deck

BACKGROUND

How BERT reads its input

Every input starts with **[CLS]** and holds one or two segments of text, each closed by **[SEP]**.
A “sentence” in the paper is any span of text (§3).

token	[CLS]	The	weather	is	nice	today	.	[SEP]	Let	'	s	go	for	a	walk	in	the	park	.	[SEP]
segment	A	A	A	A	A	A	A	A	B	B	B	B	B	B	B	B	B	B	B	B
position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

input vector = token embedding + segment embedding + position embedding

token embeddings: **28,996 × 768**

segment embeddings: **2 × 768** (A, B)

position embeddings: **512 × 768**

Adapted from Devlin et al. (2019), Figure 2, redrawn with our own sentence pair · numbers for bert-base-cased

BACKGROUND

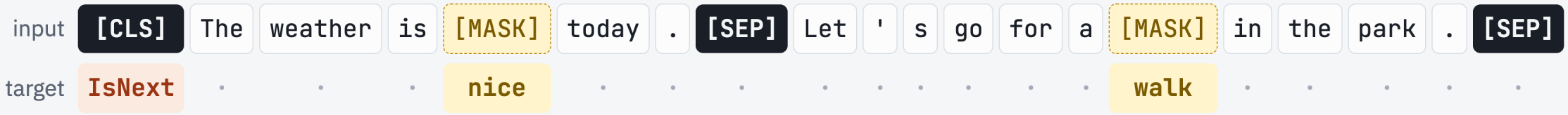
Where this format comes from: pre-training

Masked language modelling (MLM)

15% of the word pieces are chosen:
80% become [MASK], 10% a random token,
10% stay. The model recovers the originals.

Next sentence prediction (NSP)

In half of the pairs, B really follows A;
in the other half, B is a random span.
The output of [CLS] decides: IsNext or NotNext.



Both objectives at once. Each pair, with its masked tokens, goes through the model once;
the training loss is the sum of the MLM loss and the NSP loss (Appendix A.2).

Adapted from Devlin et al. (2019), Figure 1, §3.1 and Appendix A.2 · "·" = no target at this position

Q1 · WHY [SEP]? WHAT DOES IT LEARN?

Why [SEP] is there, even for one sentence

Why it exists

In pre-training, every input was a pair of segments.
NSP needs to know where A ends and B begins:
[SEP] and the segment embedding mark the boundary (§3).

Why it stays for one sentence

The model never saw an input without a closing [SEP], so fine-tuning keeps the same format. The paper calls a single text a **“degenerate text-∅ pair”** (§3.2).

```
tokenizer("The weather is nice today.")
```

`[CLS]` `The` `weather` `is` `nice` `today` `.` `[SEP]` `∅`

Segment A is your text, B is empty; the tokenizer adds [CLS] and [SEP].

WHAT DOES IT LEARN?

No loss is ever computed on the output of [SEP].

NSP uses the output of [CLS]

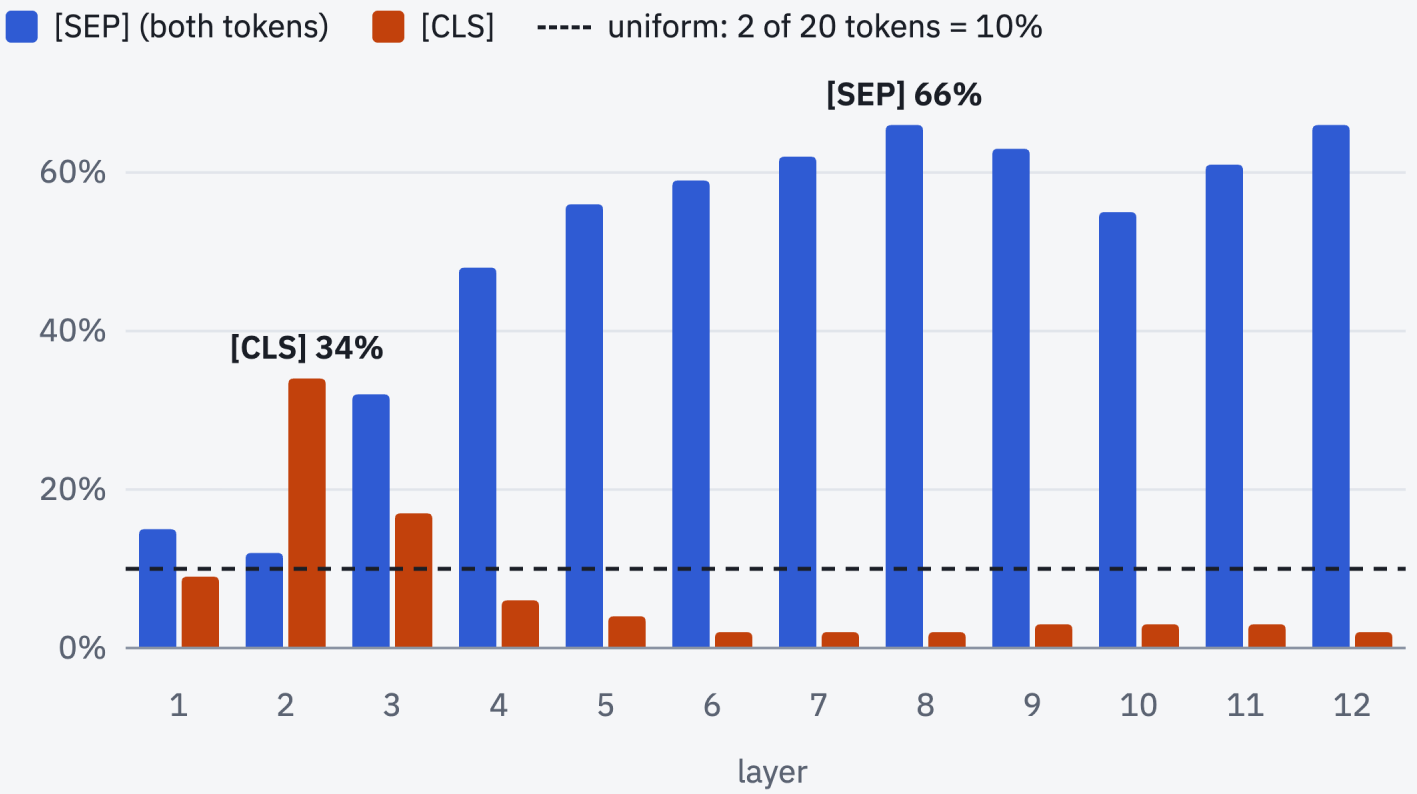
MLM predicts only the chosen 15%

Special tokens are never chosen

Its input embedding is learned (A.4), but its output has no target.
So what does the model do with it?

Q1 · WHAT DOES [SEP] LEARN?

Where the attention goes



Clark et al. (2019) found:

Over half of the attention in layers 6–10 falls on [SEP].

[SEP] attends almost only to itself and to the other [SEP].

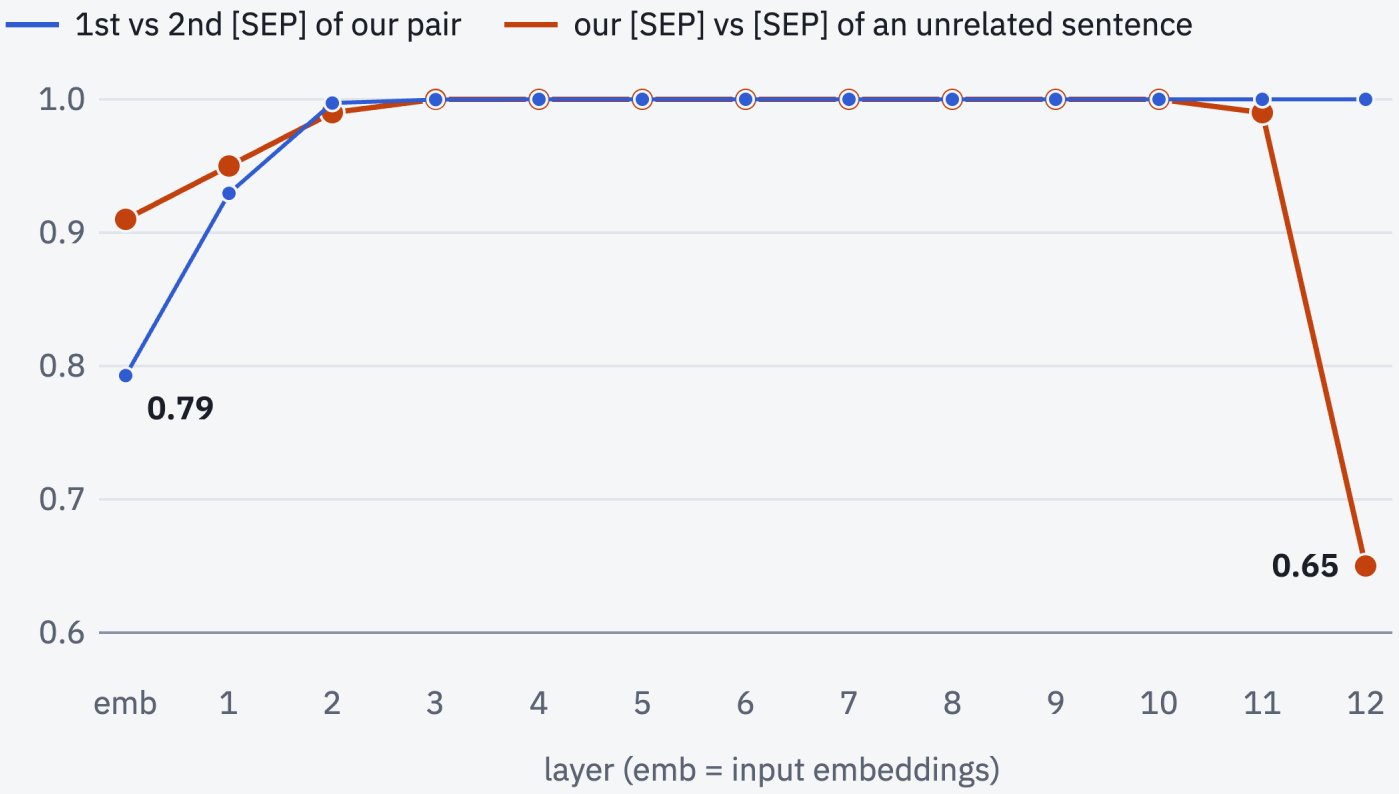
From layer 5 on, changing the attention to [SEP] barely changes the output.

Their hypothesis: a “no-op”. A head that finds nothing relevant parks its attention on [SEP].

bert-base-cased, our pair of 20 tokens, mean over 12 heads and all query positions · Clark et al. (2019), §3.2

Q1 · DO THE TWO [SEP] TOKENS DIFFER?

At the input, yes. Two layers later, no



At the input they differ: the same token embedding, but a different segment and position.

After two layers they are practically identical (cosine above 0.997).

In the middle layers it is nearly the same vector even for an unrelated text.

Only the last layer moves away (0.65).

[SEP] carries almost nothing about the text: this fits the “no-op” picture.

bert-base-cased, hidden states · unrelated sentence, fed on its own: “Interest rates rose by two percent in March.”

Q2 · WHERE CAN THE HEAD SIT?

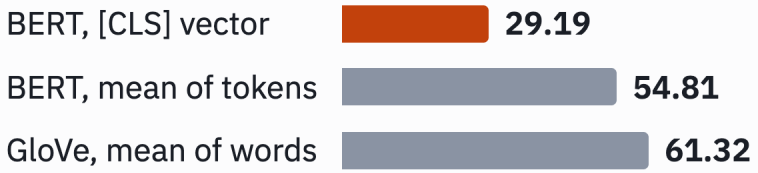
Could the head sit on [SEP] instead of [CLS]?

Technically, yes: after fine-tuning, the position hardly matters.

Fine-tuned: spam F1 with a linear head on

[CLS]	last [SEP]	mean of all tokens
0.959	0.959	0.975

Not fine-tuned: sentence similarity (STS)



[CLS] is the convention: it was pre-trained for NSP and always sits at position 0.
Without fine-tuning, though, it is a poor sentence embedding, worse than averaged GloVe (footnote 6).

Spam: bert-base-cased, one epoch, 1,000 training / 500 test SMS, one run · STS: Reimers and Gurevych (2019), Table 1, Spearman × 100

Q3 · SEVERAL SENTENCES

Do sentences have to be separated by [SEP]?

No: [SEP] separates texts, not sentences.

One text, however many sentences: one segment **A**, no [SEP] in between

```
tokenizer("The weather is nice today. Let's go for a walk.")
# [CLS] The weather is nice today . Let ' s go for a walk . [SEP]
```

Two texts: segment **A**, then [SEP], then segment **B**. Used when the task compares two texts:

```
tokenizer("The weather is nice today.", "Let's go for a walk.")
# [CLS] The weather is nice today . [SEP] Let ' s go for a walk . [SEP]
```

NLI entail, contradict or neutral?	**Paraphrase** same meaning: yes or no?	**Similarity** a score from 0 to 5	**Extractive QA** answer span in the passage

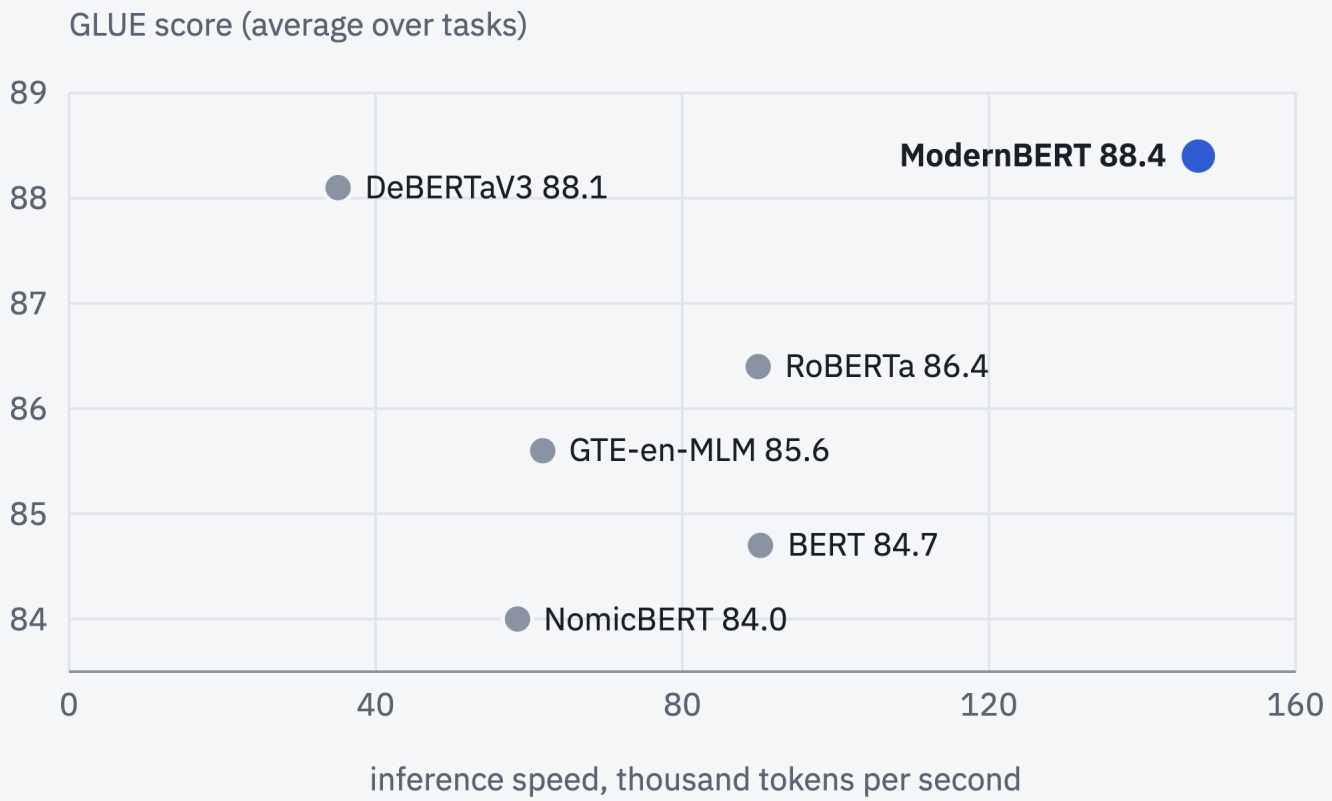
A SHORT NOTE · MODERNBERT

ModernBERT: BERT’s recipe, updated

	BERT-base (2018)	ModernBERT-base (2024)
Size	110M parameters, 12 layers	149M parameters, 22 layers
Maximum input	512 tokens	8,192 tokens
Pre-training data	3.3B words: books, Wikipedia	2 trillion tokens, including code
Objectives	MLM (15% masked) and NSP	MLM only, 30% masked
Positions	learned table of 512 positions	rotary embeddings (RoPE)
Segments	A/B segment embeddings	none: only [SEP] separates the texts
Attention	every token to every token	global in every third layer, else a window of 128
Classification head	[CLS] + pooler	mean over all tokens (set in its config)

A SHORT NOTE · MODERNBERT

More accurate and faster at the same time



88.4 vs 84.7

GLUE, ModernBERT vs BERT

147k vs 90k

tokens per second on short inputs

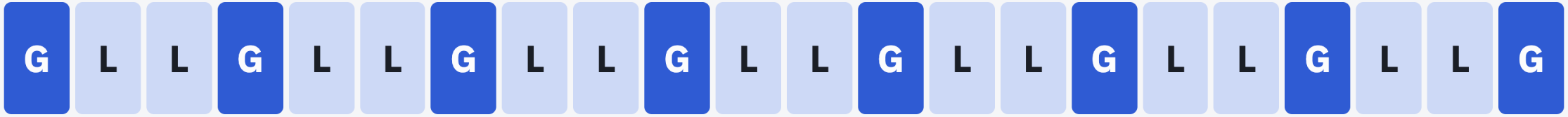
Among these base-size encoders, none beats it on either axis.

Data: Warner et al. (2024), Tables 1 and 2, licensed under CC BY 4.0 · speed: short inputs of variable length, NVIDIA RTX 4090

A SHORT NOTE · MODERNBERT

Cheaper attention: local and global layers

The 22 layers of ModernBERT-base:

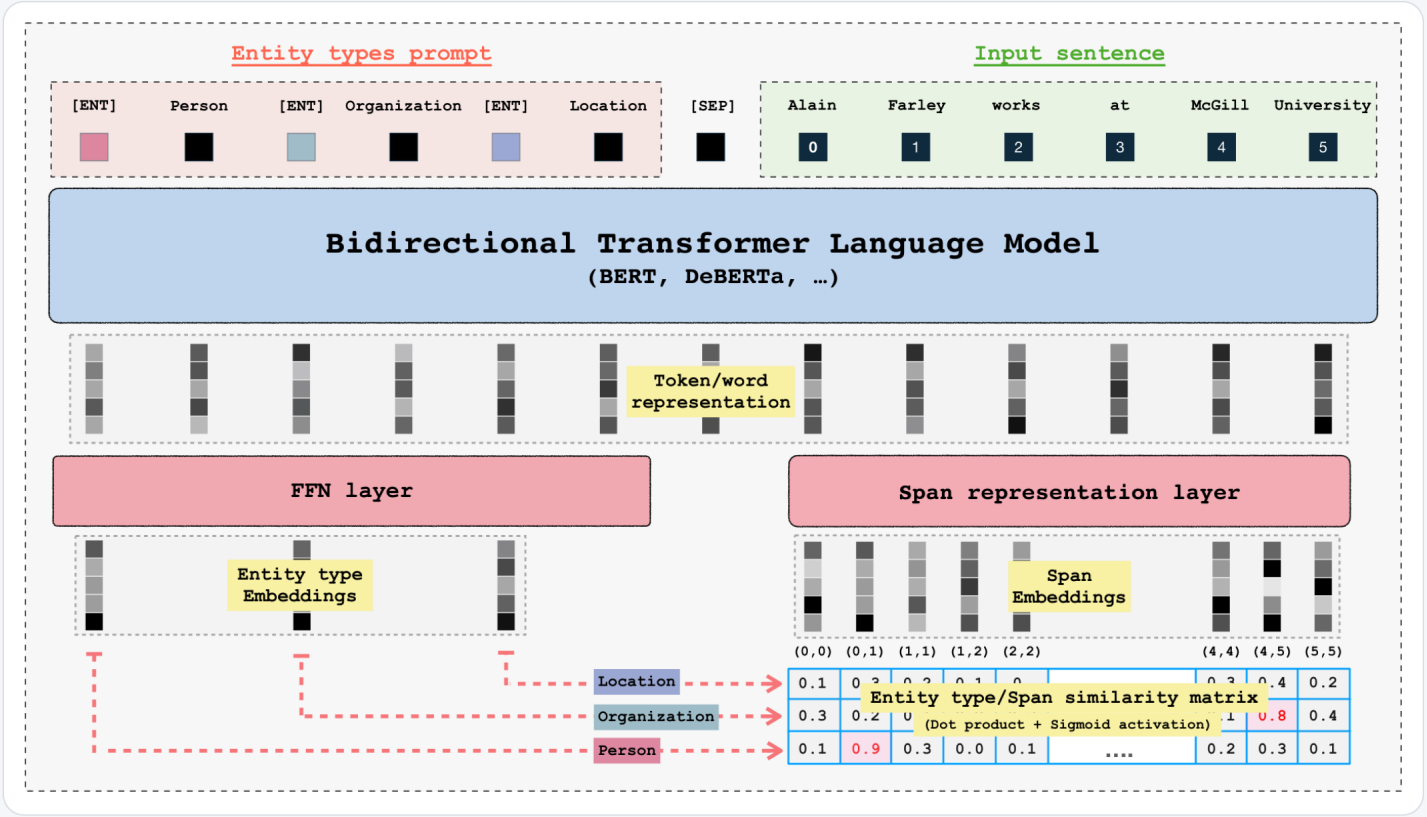


G = global attention over the whole input · **L** = local attention within a window of 128 tokens

Local layers are much cheaper on long inputs,
and information still travels across the whole text through the global ones.

NEXT · GLINER

GLiNER: matching entity types with spans



Entity type t, from its [ENT] token

```
q[t] = FFN(p[t])
```

Span (i, j), at most 12 words

```
S[i,j] = FFN(concat(h[i], h[j]))
```

Score: dot product + sigmoid

```
φ(i,j,t) = σ(S[i,j] · q[t])
```

Training: BCE over all span–type pairs

```
L = -Σ[y·log φ + (1-y)·log(1-φ)]
```

y = 1 if span (i, j) has type t, else 0

Decoding: keep φ > 0.5, then pick the best spans that do not overlap

Figure 2 from Zaratiana et al. (2024), CC BY 4.0 · formulas from §2.1–2.3, indices in square brackets

TAKEAWAYS

Why we still look at encoders

Encoders read, decoders write

One pass gives a vector for every token:
classification, NER, retrieval, reranking.

Fast, cheap and predictable

100–400M parameters, they run on a CPU;
the output is always a valid label or span.

Small and specialised can beat large

GLiNER-L (0.3B) beats ChatGPT on zero-shot NER;
a 400M encoder beats a 1B decoder on MNLI.

Still everywhere

Embeddings and rerankers in search and RAG,
safety and PII filters (Prompt Guard 2, GLiNER).

Rule of thumb: for a fixed task with labelled data, fine-tune a small model;
for open-ended output or without data, prompt a large one.

GLiNER: Zaratiana et al. (2024), Table 1 · Ettin: Weller et al. (2025), arXiv 2507.11412 · Prompt Guard 2: Meta, mDeBERTa

References

Devlin, Chang, Lee and Toutanova (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
NAACL · arXiv:1810.04805 · §3 with Figures 1 and 2, §3.1 with footnote 6, §3.2, §5.3, Appendix A

Clark, Khandelwal, Levy and Manning (2019). What Does BERT Look At? An Analysis of BERT's Attention.
BlackboxNLP · arXiv:1906.04341 · §3.2

Reimers and Gurevych (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
EMNLP · arXiv:1908.10084 · Table 1

Liu (2019). Fine-tune BERT for Extractive Summarization.
arXiv:1903.10318

Warner et al. (2024). Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder
for Fast, Memory Efficient, and Long Context Finetuning and Inference.
arXiv:2412.13663 · Tables 1 and 2

Zaratiana et al. (2024). GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer.
NAACL · arXiv:2311.08526 · §2 with Figure 2

Our own measurements and tokenizer outputs: transformers 5.16.1